

CO-1 Assessment 3

Transformer Mechanics Worksheet — SET A

Joanathan Packia Singh
192472229

Question 1 — Tokenization

5 marks

The word “UNHAPPINESS” is uncommon enough that a sub-word tokenizer would likely split it into smaller pieces rather than keep it whole as a single vocabulary entry.

(a) A reasonable 3-piece split

A linguistically motivated split breaks the word along its morpheme boundaries — the smallest meaningful units that make up the word:

Piece	Type	Role in the word
UN	Prefix	Negation — flips the meaning of what follows
HAPPI	Root / stem	Carries the core meaning (“happy”, spelling adjusted for suffixing)
NESS	Suffix	Turns an adjective into an abstract noun (state/quality of)

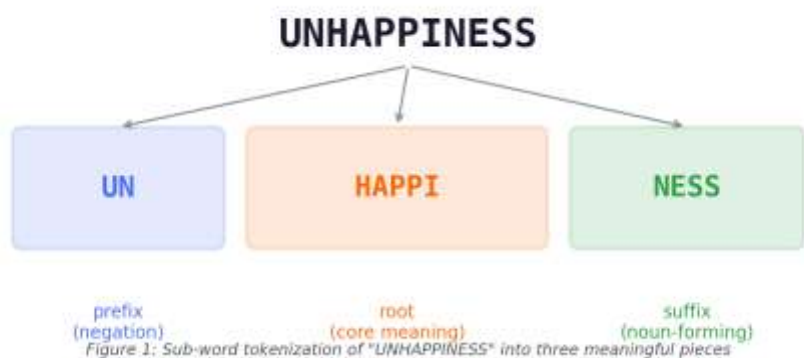


Figure 1: “UNHAPPINESS” decomposed into prefix + root + suffix, mirroring how a Byte-Pair-Encoding (BPE) or WordPiece tokenizer would greedily merge frequent sub-word units.

This isn’t the only valid split — a real BPE/WordPiece vocabulary built from statistics rather than linguistics might instead produce something like “UN” + “HAPP” + “INESS”, or even “UNHAP” + “PIN” + “ESS” depending on which character sequences were most frequent in the training corpus. What matters for full marks is that the three pieces (i) concatenate back to the original word, (ii) are plausible units the tokenizer’s merge rules could actually produce, and (iii) are justified with a reason (frequency, morphology, or byte-pair merge order).

(b) Why sub-word tokenization helps

Sub-word tokenization lets a model represent an unseen word compositionally, as a combination of smaller pieces it has already learned during training, instead of needing to have memorized that exact whole word. Because “UN-”, “HAPPI-”, and “-NESS” each appear in many other common words (“undo”, “happy”, “kindness”), the model can reuse the meaning and behavior it already learned for those pieces and combine them to infer the meaning of the novel whole word, rather than treating “UNHAPPINESS” as a completely unknown, out-of-vocabulary token.

- Avoids the out-of-vocabulary (OOV) problem: whole-word tokenizers must map anything unseen to a single generic <UNK> token, destroying all information.
- Keeps the vocabulary size manageable: a fixed set of a few tens of thousands of sub-word units can reconstruct virtually any word, instead of needing millions of whole-word entries.
- Generalizes across morphology: the model learns “UN-” means negation once, and that knowledge transfers to “unhappy”, “unfair”, “undo”, “unhappiness”, etc. without separate training for each.

Question 2 — Embeddings

5 marks

Consider the word pairs “cat” & “kitten”, and “cat” & “car”.

(a) Which pair has higher cosine similarity?

The pair “cat” and “kitten” would be expected to have the higher cosine similarity.

Embedding vectors are learned so that words appearing in similar contexts end up close together in vector space. “Cat” and “kitten” are semantically related (a kitten is literally a young cat) and are largely interchangeable in many sentence contexts (“the ___ chased the mouse”, “my ___ is sleeping on the sofa”), so the training signal pushes their vectors to point in a very similar direction — giving a small angle between them and hence a cosine similarity close to 1.

“Cat” and “car”, by contrast, only share surface-level spelling (both start with “ca”) but belong to entirely different semantic categories — one is a living animal, the other is a mechanical vehicle. They rarely appear in the same contexts, so their embedding vectors end up pointing in noticeably different directions, giving a lower cosine similarity despite the superficial string overlap. This nicely illustrates that embeddings capture meaning, not spelling.

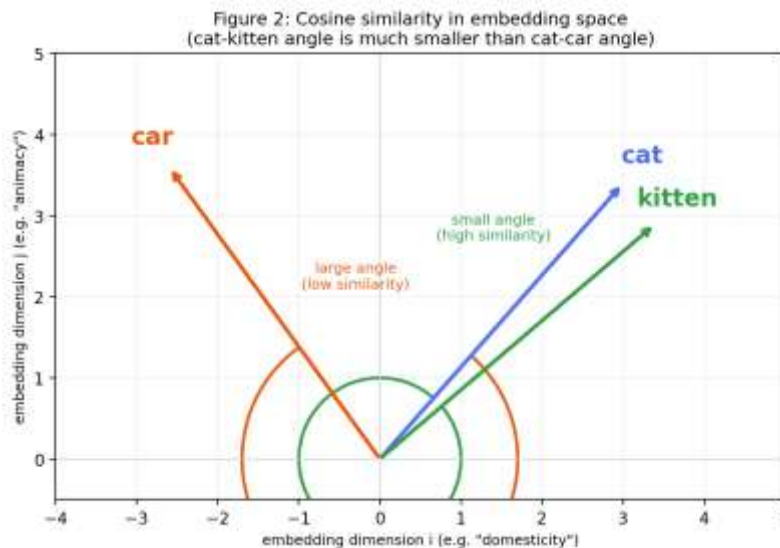


Figure 2: In embedding space, the angle between “cat” and “kitten” is much smaller than between “cat” and “car”, so cosine similarity (which depends only on the angle) is higher for the semantically related pair.

(b) What an embedding represents

An embedding is a dense, real-valued vector (typically a few hundred to a few thousand dimensions) that represents a word's meaning as a position in a continuous, learned vector space. Each dimension doesn't usually correspond to one clean human-readable feature, but collectively the geometry of the space captures semantic and syntactic relationships: words with similar meanings or usage patterns cluster near each other, and directions in the space can even encode relationships (the classic example being that $\text{vector}(\text{"king"}) - \text{vector}(\text{"man"}) + \text{vector}(\text{"woman"})$ lands close to $\text{vector}(\text{"queen"})$).

This is fundamentally different from a one-hot vector, which represents a word as a single sparse vector the size of the entire vocabulary, with a 1 in the position for that word and 0s everywhere else. Key differences:

- Dimensionality: a one-hot vector has as many dimensions as the vocabulary (often tens of thousands); an embedding is dense and compact (e.g. 300–4096 dimensions).
- Similarity: any two distinct one-hot vectors are equally and maximally dissimilar (orthogonal) by construction — “cat” and “kitten” are just as “different” as “cat” and “car”. Embeddings instead encode graded, meaningful similarity.
- Learnability: one-hot vectors are fixed, arbitrary identifiers; embeddings are learned from data so that geometric closeness reflects real semantic or functional closeness.

Question 3 — Self-Attention Worked Example

5 marks

Sentence: “The trophy didn't fit in the suitcase because it was too big.”

(a) What does “it” refer to?

Answer: **“trophy”** (circled, as opposed to “suitcase”)

Because the trophy didn't fit in the suitcase, common-sense reasoning tells us the trophy must be the object that is too big — the suitcase is, if anything, comparatively too small. So “it” should attend most strongly to “trophy”.

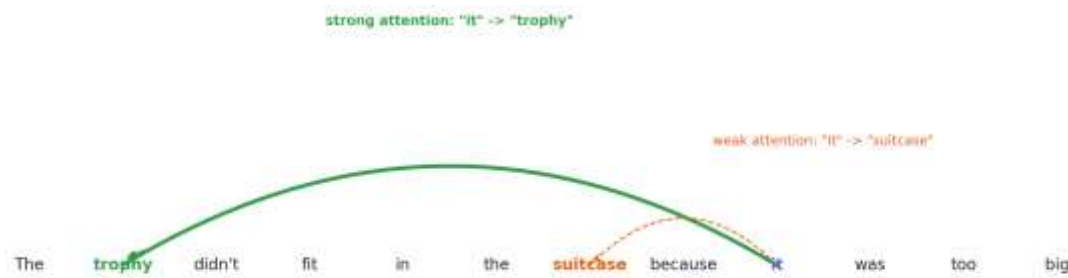


Figure 3: Self-attention weights for “it” in “The trophy didn't fit in the suitcase because it was too big.”

Figure 3: The attention weight from “it” to “trophy” (solid green arc) should be much stronger than the weight from “it” to “suitcase” (dashed orange arc).

(b) Why self-attention, not word-order rules, is needed

A simple word-order or proximity rule (such as “pronouns refer to the nearest preceding noun”) would incorrectly resolve “it” to “suitcase”, since “suitcase” sits immediately before “it” in the sentence while “trophy” is further away. Resolving the reference correctly requires combining information from the whole sentence, especially the

word “big” at the end, with world knowledge about size and containment (a bigger object cannot fit inside a smaller one).

Self-attention is exactly the mechanism that allows this: every word’s representation is updated by looking at, and weighting, every other word in the sentence — regardless of distance — based on learned relevance rather than fixed position. This lets the model dynamically route information from “big” and the “didn’t fit” construction back to “it”, correctly linking it to “trophy” even though “suitcase” is closer in the raw sequence. This exact sentence (from the Winograd Schema Challenge) is a classic test case precisely because it defeats simple positional heuristics and requires genuine contextual/semantic reasoning.

Question 4 — Query, Key, Value

5 marks

Self-attention computes, for every token, a new representation built from a weighted combination of all tokens in the sequence. This weighting is computed using three learned projections of each token’s embedding: the Query, Key, and Value vectors.

Vector	Intuition	Role in the computation
Query (Q)	“What am I looking for?” — represents the current token’s question about the rest of the sequence	Compared against every Key to measure relevance
Key (K)	“What do I contain / advertise?” — represents what each token has to offer	Dot-producted with the Query to produce a raw attention score
Value (V)	“What information do I actually pass on?” — the substantive content of each token	Combined (weighted sum) using the normalized attention scores to build the output

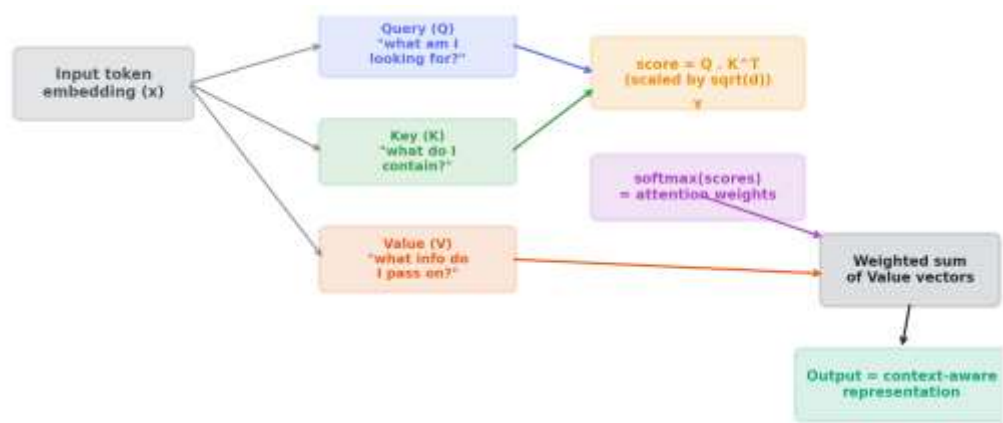


Figure 4: How Query, Key, and Value vectors combine to produce an attention output

Figure 4: The *Query* of a token is compared against the *Keys* of every token (dot product), scaled and passed through softmax to get attention weights, which then weight a sum over the *Value* vectors to produce the output.

How they combine

- Step 1 — Score: for a given token, its Query vector is dot-producted with the Key vector of every other token (including itself), producing one raw similarity score per token pair.
- Step 2 — Scale: the scores are divided by the square root of the key dimension ($\sqrt{d_k}$) to keep gradients stable during training.
- Step 3 — Normalize: a softmax is applied across all the scaled scores for that Query, converting them into attention weights that are non-negative and sum to 1.
- Step 4 — Aggregate: the output for that token is the weighted sum of every token's Value vector, using those softmax weights — so tokens whose Key strongly matched the Query contribute the most Value content to the result.

In short: Query and Key together decide “how much attention to pay to what”, while Value supplies “the actual content that gets passed along” once that attention distribution has been decided. This is what allows the model, in the trophy/suitcase example above, to let the representation of “it” be updated mostly by the Value vector of “trophy”.
